

```

g@t3keeper@ubuntu-test:/var/www/html$ sudo crontab -e
no crontab for root - using an empty one

Select an editor. To change later, run "select-editor".
 1. /bin/ed
 2. /bin/nano <---- easiest
 3. /usr/bin/vim.basic
 4. /usr/bin/vim.tiny

Choose 1-4 [2]: _

```

This first two parameters allow you to set the time. So you have minutes(0-59) and then hours(0-23). Then there's the day(1-31) of the month, month(1-12) and day of the week(0-6, 0 being Sunday). And lastly the command to be executed. We want to run the renewal command every week on a Monday at 2:30 AM. So we'll be using:

```
30 2 * * 1 /usr/bin/letsencrypt renew >> /var/log/le-renew.log
```

```

GNU nano 2.5.3      File: /tmp/crontab.Vf6Bb/crontab      Modified
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
30 2 * * 1 /usr/bin/letsencrypt renew >> /var/log/le-renew.log

```

You should now save and exit crontab. We've also set a log file to be created for each instance of the cron job to be saved at /var/log/le-renew.log.

The alternate way

Sometimes, if your domain is behind a firewall or if you have a CDN setup, then the usual method will not work. This is because the Let's Encrypt authentication server needs to per-

form a challenge with the IP address associated with your domain. And with a CDN involved, the authentication server will see the challenge request coming from a different IP address. In this case, we need to use the webroot method. This method simply creates a temporary file in the root folder of your domain which the Let's Encrypt authentication server can read from to verify domain ownership. To do that, use the following command:

```
certbot certonly --webroot -w /var/www/yourdomainname/ -d www.yourdomainname.com
```

The Let's Encrypt server will make an HTTP request to find this temporary file at your www.yourdomainname.com and the certbot will create that temporary file at /var/www/yourdomainname/ in your server. Ensure that you create the temporary file in the root directory and nowhere else. Hopefully, you'll now have HTTPS enabled via the webroot method.

Conclusion

HTTPS is the future of the web, there's no denying that. It protects the integrity of your website and ensures the privacy and security of visitors coming to your site. When we've entered an era of needless mass surveillance, the web has started to become less free and subject to censorship. The latter has been used by governments to suppress their people and is still being used to do the same in many regions across the world. HTTPS may not be the one true solution to this problem but it sure does go a long way towards alleviating the privacy issue plaguing the internet.

 **Secure** | <https://www.yoursite.com>

Let's Encrypt, through its service has made the process of obtaining SSL certificates ridiculously easy, to the point the even absolute beginners can implement HTTPS on their domains. Moreover, it's free. So what's holding you back from getting your own SSL certificate? >

*Nodes of Interest



*Data Encryption

>> Confused about the basics of encryption? Give this course a shot to get back on track.

<http://dgjit.in/DataEncryp>



*Secure Rancher apps

>> Here's how to secure your Rancher web app with Let's Encrypt on Ubuntu 16.04.

<http://dgjit.in/SecureRancher>



*Secure your website

>> A feature that covers the major areas that you need to secure if you have your own website.

<http://dgjit.in/SecureWb>